

MODULE 4.2: AUTOENCODERS, CNN, RNN

Contents

Convolutional Neural Network (CNN)	1
Recurrent Neural Network (RNN)	3
Graph Neural Networks (GNNs)	5
Autoencoders (AE)	5
Variational Autoencoders (VAE)	6

Convolutional Neural Network (CNN)

- A **Convolutional Neural Network (CNN)** is a deep learning model designed primarily for **image and spatial data**.
- It captures **spatial hierarchies** — patterns like edges, textures, and shapes — using **convolution operations** instead of dense connections.
- It's inspired by how the human visual cortex processes images.

Architecture

A typical CNN has the following layers:

Layer	Purpose
Input Layer	Takes image data (e.g., 28×28 pixels for MNIST).
Convolutional Layer	Applies filters (kernels) to extract features such as edges or textures.
Activation Function (ReLU)	Adds non-linearity to feature maps.
Pooling Layer	Reduces spatial size (e.g., max pooling) → lowers computation and captures invariance.
Fully Connected Layer	Connects all neurons to perform classification or regression.
Output Layer	Produces probabilities (via Softmax) or numeric output.

Mathematical Operation

For a convolution layer:

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) \cdot K(m, n)$$

where

- X: input image,
- K: kernel/filter,
- Y: output feature map.

Key Concepts

Concept	Description
Local Connectivity	Each neuron connects only to a small region of the input (receptive field).
Weight Sharing	Same filter slides across the image — fewer parameters.
Feature Hierarchy	Lower layers → simple edges; higher layers → complex shapes.

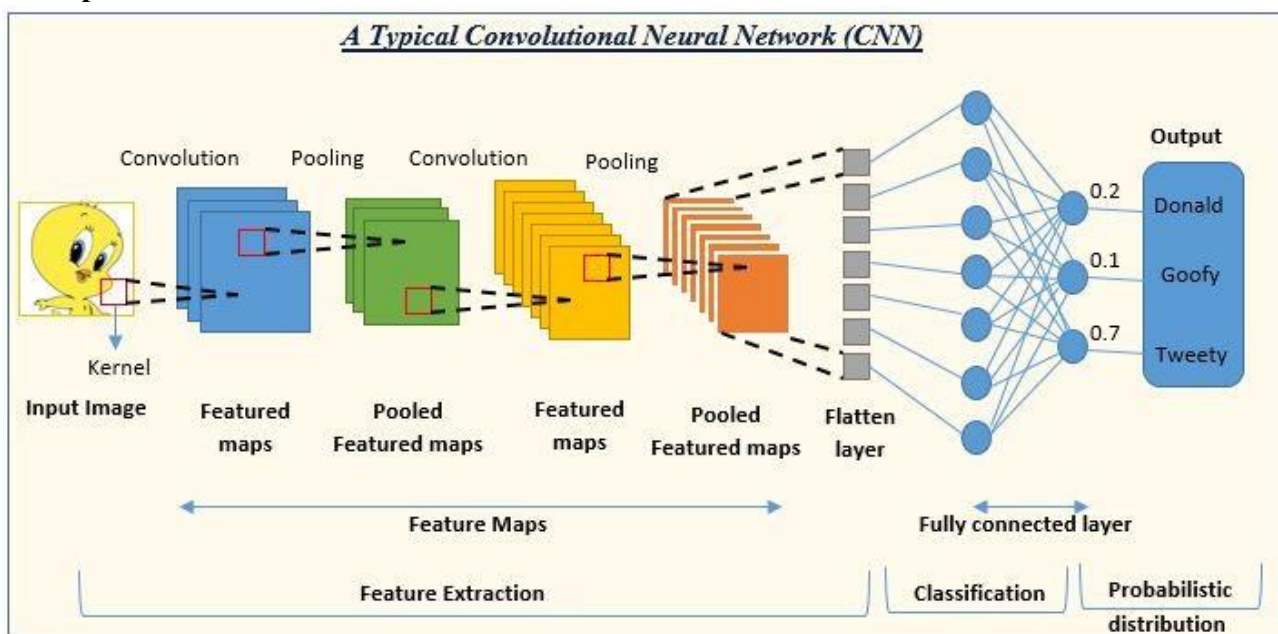
Advantages

- Captures spatial features effectively
- Requires fewer parameters than dense networks
- Invariant to translation, rotation, scaling
- Excellent for image and video data

Applications

- Image classification (e.g., object, face recognition)
- Medical imaging (tumor or disease detection)
- Self-driving car vision systems
- Video and scene understanding

Example



The CNN process is divided into three major phases: **Feature Extraction**, **Classification**, and **Probabilistic Distribution**.

Phase 1: Feature Extraction (Convolutional Base)

This phase automatically learns hierarchical representations of the input image.

1. **Input Image:** The process begins with the input image (in this case, the cartoon character 'Tweety').
2. **Convolution Layer:** This is the core operation. A small filter, called a **Kernel** (shown with dashed lines and a red box), slides across the entire input image.
 - The kernel performs a dot product between its values and the corresponding input pixels.
 - This operation detects specific local features (like edges, corners, or textures).
 - The output of this layer is a set of **Feature Maps** (shown in blue and yellow), where each map represents the presence of a specific feature across the image.
3. **Pooling Layer:** This layer is introduced after the convolution layer to **reduce the spatial size** of the feature maps, which helps reduce computational cost and control overfitting.
 - It operates by taking a small window (e.g., 2×2) and calculating the maximum (Max Pooling) or average (Average Pooling) value within that window.

- The output is the **Pooled Feature Maps** (shown in green and orange), which retain the most important feature information while being smaller.
4. **Repetition:** The Convolution and Pooling layers are often **repeated multiple times** (e.g., Convolution → Pooling → Convolution → Pooling) to build deeper, more abstract feature representations. The early layers learn simple features (edges), and deeper layers learn complex features (eyes, beaks, shapes).

Phase 2: Classification (Fully Connected Head)

Once the features are extracted, the model needs to use them to make a final decision.

1. **Flatten Layer:** The 3D feature maps (the output of the Feature Extraction phase) are converted into a single, long **1D vector**. This transformation is necessary to feed the data into the traditional layers of the network.
2. **Fully Connected Layer:** This layer is a standard Multi-Layer Perceptron (MLP) where every neuron from the previous layer is connected to every neuron in the next layer.
 - This layer learns **non-linear combinations** of the extracted features to determine the final high-level categories.

Phase 3: Output (Probabilistic Distribution)

1. **Output Layer:** The final layer is also a fully connected layer, but the number of neurons equals the number of target classes (in this image: Donald, Goofy, Tweety).
2. **Probabilistic Distribution:** A function like **Softmax** is applied to the output neurons. This converts the raw scores into a **probability distribution** where all values sum to 1.0.
 - In the example shown: Donald has a probability of 0.2, Goofy has 0.1, and Tweety has 0.7.
 - **Final Prediction:** The model correctly identifies the image as belonging to the class with the highest probability, which is **Tweety (0.7)**.

Recurrent Neural Network (RNN)

- A **Recurrent Neural Network (RNN)** is designed for **sequential or time-series data**.
- Unlike CNNs, RNNs have **feedback connections**, allowing them to remember **previous inputs** and capture **temporal dependencies**.
- In short, RNNs have “memory.”

Architecture

Layer	Purpose
Input Layer	Accepts sequential data (e.g., words, sensor readings).
Hidden Layer(s)	Processes current input and previous hidden state.
Output Layer	Produces output at each time step or final output.

Mathematical Representation

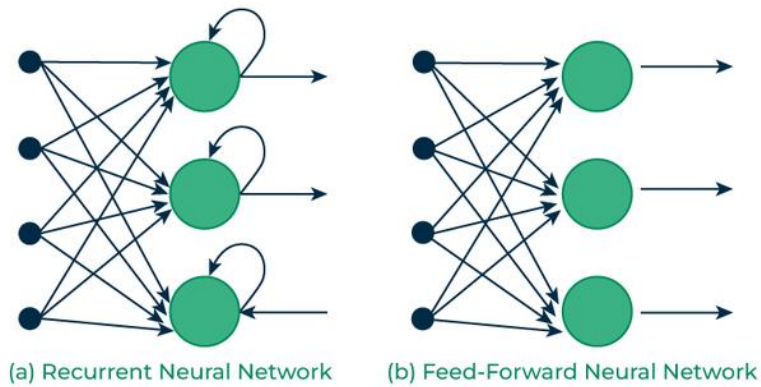
At time step t :

$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \quad y_t = g(W_{hy}h_t + b_y)$$

where

- x_t : input at time t
- h_t : hidden state (memory)
- y_t : output
- W_{xh}, W_{hh}, W_{hy} : weight matrices

- f, g : activation functions (usually tanh or ReLU)



Key Property

- The **hidden state h_t** carries information forward, allowing RNNs to **learn temporal context**.
- Training uses **Backpropagation Through Time (BPTT)** to handle sequences.

Limitations

- Struggles with **long-term dependencies** due to **vanishing/exploding gradients**.
- Training is slow for long sequences.
- Solved by advanced variants like **LSTM (Long Short-Term Memory)** and **GRU (Gated Recurrent Unit)**.

Advantages

- Handles sequence data naturally
- Remembers context and order of inputs
- Works for variable-length input sequences

Applications

- **Natural Language Processing (NLP)**: text generation, translation, sentiment analysis
- **Speech recognition and synthesis**
- **Stock price prediction and weather forecasting**
- **Time-series anomaly detection**

Comparison Between CNN and RNN

Aspect	CNN	RNN
Data Type	Spatial (e.g., images)	Sequential / temporal (e.g., text, speech)
Connection	Feedforward (no memory)	Recurrent (has memory via feedback)
Operation	Convolution + Pooling	Recurrence over time steps
Feature Extraction	Spatial / local features	Temporal / contextual features
Training	Standard backpropagation	Backpropagation Through Time (BPTT)
Parallelization	Easy	Harder (sequential dependence)
Example Use	Object recognition, vision tasks	Language modeling, time-series forecasting

Graph Neural Networks (GNNs)

Graph Neural Networks (GNNs) are a family of deep learning models designed to operate directly on data structured as **graphs**. Unlike traditional neural networks which process sequential or grid data, GNNs leverage the connectivity (edges) and features (nodes) within a graph structure.

Key Concept: Message Passing

The core mechanism of GNNs is **Message Passing**, which is an iterative process where nodes update their feature representations (embeddings) based on their neighbors' representations:

1. **Message Generation:** Each node aggregates information ("messages") from its immediate neighbors.
2. **Aggregation:** The node pools all received messages (e.g., by summing, averaging, or using an attention mechanism).
3. **Update:** The node combines the aggregated message with its own previous state to compute a new, updated feature vector.

This process is repeated for several layers, allowing information to propagate across the entire graph. After K iterations, a node's embedding contains information from its K -hop neighborhood.

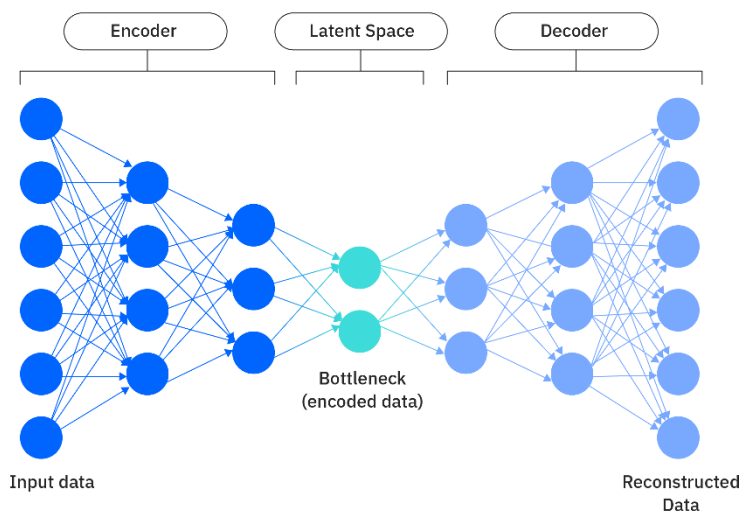
Applications

GNNs are effective for tasks where relationships are key:

- **Node Classification:** Predicting the label of a node (e.g., classifying a user in a social network).
- **Link Prediction:** Predicting the existence of a link between two nodes (e.g., recommending a friendship or a new drug-target interaction).
- **Graph Classification:** Classifying the entire graph (e.g., identifying a toxic molecule structure).

Autoencoders (AE)

- An **Autoencoder** is a type of **unsupervised neural network** that learns to **reconstruct its input**.
- It compresses the input into a **lower-dimensional latent space** (encoding) and then reconstructs it back (decoding).
- **Goal** → Learn a compact representation of data that captures its most important features.



Purpose

- Dimensionality reduction (like PCA but nonlinear)
- Feature learning / representation learning
- Noise removal (Denoising Autoencoders)
- Data generation and anomaly detection

Architecture

An Autoencoder consists of **three main parts**:

Component	Function
Encoder	Maps the input x to a lower-dimensional latent vector z .
Latent Space (Bottleneck)	Represents the compressed or encoded data (learned features).
Decoder	Reconstructs the input $x^{\wedge}$ from z .

Limitation

- Regular autoencoders **just memorize training data** instead of learning a smooth latent distribution.
- Not good at **generating new, diverse samples**.

This leads us to the **Variational Autoencoder (VAE)**.

Variational Autoencoders (VAE)

- A **Variational Autoencoder (VAE)** is a **generative model** that learns not only to reconstruct input but also to **model the underlying probability distribution** of the data.
- It can **generate new samples** similar to the training data.
- Key Idea**
 - Instead of encoding each input into a fixed vector z , a VAE encodes it as a **probability distribution** — usually a Gaussian $N(\mu, \sigma^2)$.
 - This adds randomness and allows **sampling** new z vectors to generate new data.

Architecture

Component	Function
Encoder	Outputs two vectors: mean μ and standard deviation σ .
Sampling Layer	Samples $z = \mu + \sigma \odot \epsilon$, where $\epsilon \sim N(0, 1)$.
Decoder	Takes sampled z and reconstructs $x^{\wedge}$.

Loss Function

VAE loss combines two terms:

$$L = \underbrace{\mathbb{E}_{q(z|x)}[\log p(x|z)]}_{\text{Reconstruction Loss}} - \underbrace{D_{KL}(q(z|x)||p(z))}_{\text{KL Divergence (Regularization)}}$$

Term	Meaning
Reconstruction Loss	Measures how accurately the decoder reconstructs input (similar to AE loss).
KL Divergence	Ensures the latent distribution ($q(z)$

Advantages of VAE

- Learns **smooth, continuous latent spaces**.
- Can **generate new, realistic samples**.
- Acts as both **compressor** and **generator**.
- Useful in image synthesis, anomaly detection, and data augmentation.

Aspect	Autoencoder (AE)	Variational Autoencoder (VAE)
Type	Deterministic	Probabilistic
Latent Representation	Fixed vector $z=f(x)$	Distribution $z \sim N(\mu, \sigma^2)$
Sampling	Not possible (no randomness)	Can sample new data points
Output	Reconstructs inputs	Reconstructs + generates new samples
Loss Function	Reconstruction error only	Reconstruction + KL Divergence
Use Case	Compression, denoising	Generative modeling, anomaly detection